End-to-End Documentation

Project Overview

This document explains the complete integration of a **Spring Boot backend** with a **React + Vite (JavaScript) frontend** for **User Registration and Login**. It covers backend APIs, frontend setup, CORS & Spring Security configuration, common errors, and verification steps.

Tech Stack

Backend

- Java 17+
- Spring Boot
- Spring Web
- Spring Data JPA
- Spring Security (if enabled)
- MySQL / H2

Frontend

- React
- Vite
- JavaScript
- Axios
- React Router DOM

Backend API Specification

Base URL

http://localhost:8080/api

Register API

Endpoint

POST /register

Request Body (JSON)

```
{
  "username": "Karan",
  "email": "karan@gmail.com",
  "password": "1234"
}
```

Response (Text)

User Registered Successfully

Status Code

200 OK

Login API

Endpoint

POST /login

Request Body (JSON)

```
{
  "email": "karan@gmail.com",
  "password": "1234"
}
```

Response (Text)

Login Successful

OR

Invalid Email or Password

Backend Controller Example

```
@CrossOrigin(origins = "http://localhost:5173")
@RestController
@RequestMapping("/api")
public class AuthController {

  @PostMapping("/register")
  public ResponseEntity<String> register(@RequestBody User user) {
    userRepository.save(user);
    return ResponseEntity.ok("User Registered Successfully");
  }

  @PostMapping("/login")
  public ResponseEntity<String> login(@RequestBody LoginRequest req) {
    boolean success = authService.login(req);
    return success
      ? ResponseEntity.ok("Login Successful")
      : ResponseEntity.status(401).body("Invalid Email or Password");
  }
}
```

Spring Security Configuration (IMPORTANT)

Why Needed

- Browsers enforce **CORS**
- Spring Security blocks requests using **CSRF** by default
- Postman works but browser fails unless configured

SecurityConfig.java

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
  http
    .csrf(csrf -> csrf.disable())
    .cors(cors -> {})
    .authorizeHttpRequests(auth -> auth
      .requestMatchers("/api/register", "/api/login").permitAll()
      .anyRequest().authenticated()
    );

  return http.build();
}
```

Frontend Setup

Create React Vite App

```
npm create vite@latest login-frontend -- --template react
cd login-frontend
npm install
npm install axios react-router-dom
npm run dev
```

Frontend URL:

http://localhost:5173

Frontend Folder Structure

```
src/
 ├── api/
```

```
│   └── api.js
├── pages/
│   ├── Register.jsx
│   ├── Login.jsx
│   └── Dashboard.jsx
├── App.jsx
├── main.jsx
└── index.css
```

Axios Configuration

src/api/api.js

```
import axios from "axios";

const api = axios.create({
  baseURL: "http://localhost:8080/api",
  headers: {
    "Content-Type": "application/json",
  },
});

export default api;
```

Register Page

```
await api.post("/register", {
  username,
  email,
```

password,

});

Login Page

await api.post("/login", {

email,

password,

});

Routing Configuration

<Routes>

<Route path="/" element={<Login />} />

<Route path="/register" element={<Register />} />

<Route path="/dashboard" element={<Dashboard />} />

</Routes>

Common Issues & Fixes

1. Registration Failed (but Postman works)

Cause: CORS or CSRF
Fix:

- Add @CrossOrigin
- Disable CSRF in Spring Security

---

2. 403 Forbidden

Cause: Spring Security blocking endpoint
Fix:

.requestMatchers("/api/register", "/api/login").permitAll()

---

3. 415 Unsupported Media Type

Cause: Missing JSON header
Fix:

Content-Type: application/json

---

4. Entity Field Mismatch

Frontend JSON fields **must match entity variables** exactly:

private String username;

private String email;

private String password;

Verification Checklist

- ✅ Backend running on port 8080
- ✅ Frontend running on port 5173
- ✅ Register works in Postman
- ✅ Register works in browser
- ✅ Login works in browser

Final Result

- React frontend successfully communicates with Spring Boot backend
- User registration and login work in browser
- CORS and CSRF issues resolved
- Ready for JWT, roles, and production security

Next Enhancements (Optional)

- JWT Authentication

- Role-based dashboards
- Password hashing (BCrypt)
- Protected routes
- Logout
- UI with Bootstrap / Tailwind

GitHub: https://github.com/karantondeprofessional-collab/login-frontend